AWS Level 3 iteration 2 terminaldokumentation

Claudio Verdi | APL | 23 till 24 oktober 2026

Jag skapade en Ubuntu-instans med Terraform och använde user-data för att automatiskt uppdatera paket och installera samt starta Nginx. Därefter publicerade jag webbplatsen manuellt via SSH. HTTP-tester och ett innehållstest lyckades. Instansen togs bort efter testerna och slutstatusen verifierades som terminated.

Miljö och resurser

Egenskap	Värde
Region	eu-north-1
Terraform / AWS provider	1.16.3 / hashicorp/aws 6.70.0
Instance ID	i-08bb6ecd7cad083f4
Instanstyp	t3.micro
AMI	ami-06fb1802e2b757f09
OS	Ubuntu 24.04.5 LTS x86_64
Subnet	subnet-05d4dca4820d2eb94
Security group	sg-0fb87d0f08a213466
Nyckelpar	devops.school.level3.tabish-admin
Tidigare publik IP	13.53.35.150
Privat IP	172.31.19.103
Nginx / Git	1.24.0 / 2.43.0
Rötdisk	8 GiB gp3 / encrypted / delete_on_termination

Referens: Level3 Iteration2 (1)(2)(1).sh. Guidens Amazon Linux-kommandon anpassades till Ubuntu och den faktiska filstrukturen.

Förberedelser och user data

Jag skapade ~/aws-devops-level3-iteration2 och kopierade fem konfigurationsfiler från iteration 1. Terraform-state kopierades inte. Jag återanvände region, Ubuntu-AMI, subnät, säkerhetsgrupp och nyckelpar från föregående iteration.

user-data.web.sh använder apt-get och systemctl. Skriptet körs som root av cloud-init vid första starten. bash -n kontrollerade skriptets syntax utan att köra installationen lokalt.

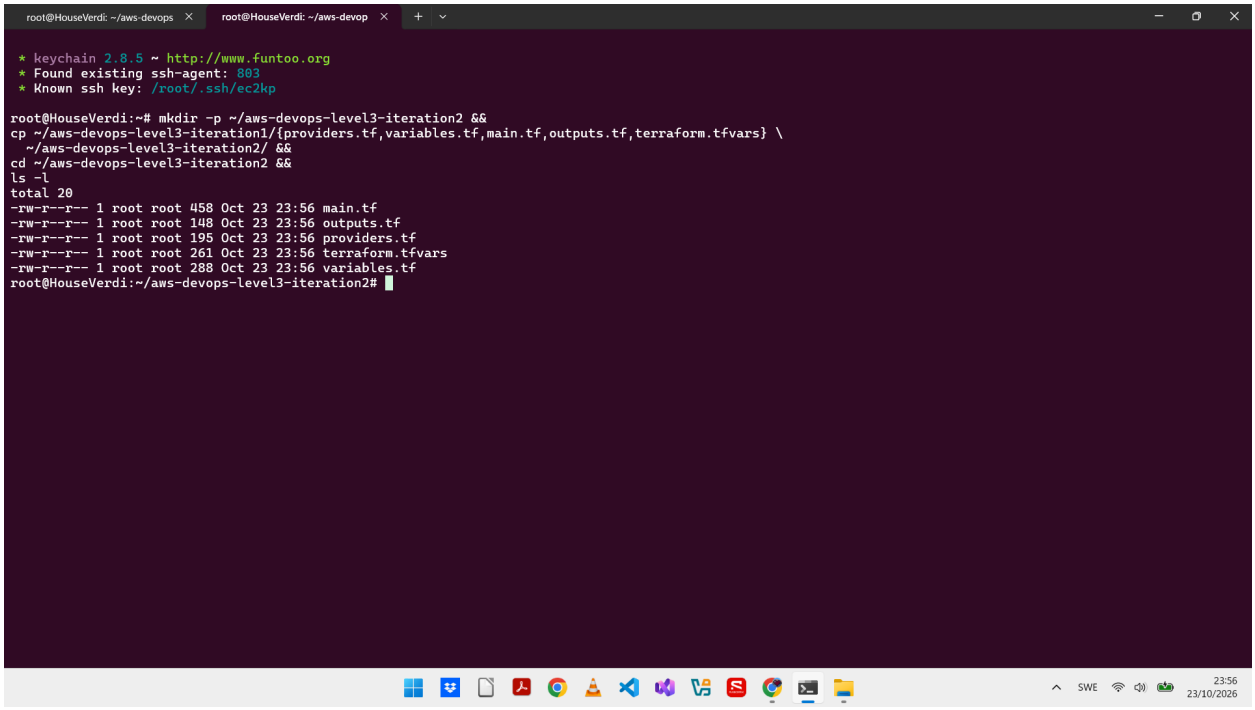


Bild 1. Fem konfigurationsfiler i en separat katalog

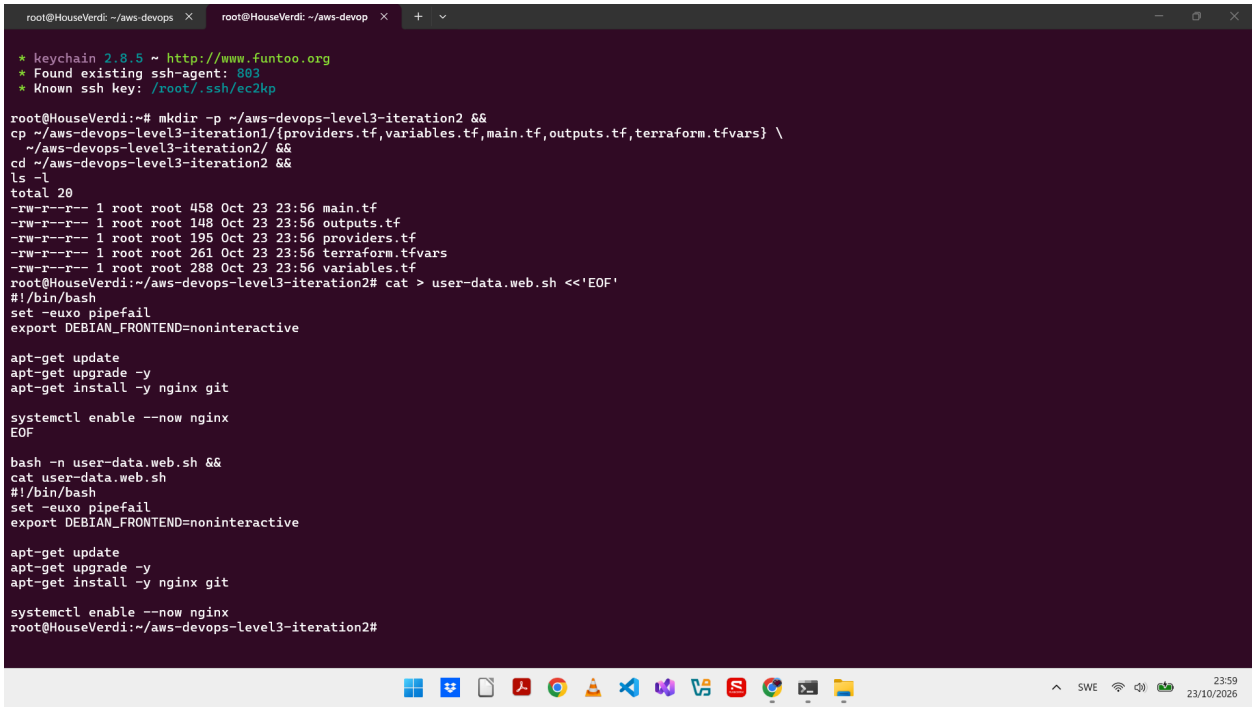


Bild 2. User-data-skriptet skapat och syntaxkontrollerat

Terraform konfiguration och validering

Jag kopplade user_data till skriptfilen och satte user_data_replace_on_change=true. Kravet på IMDSv2, publik IP och krypterad rotdisk behölls. Terraform hanterade endast EC2-instansen; nätverk och nyckelpar fanns redan.

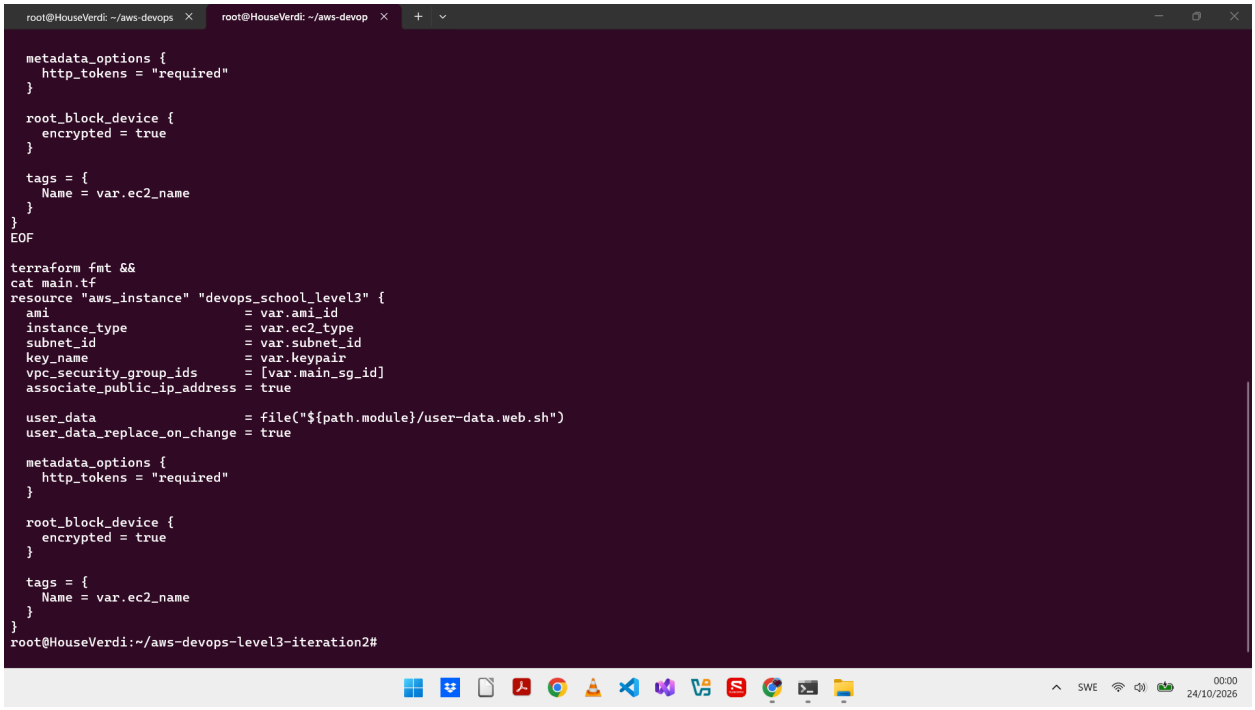


Bild 3. main.tf med user_data och ersättning vid ändringar

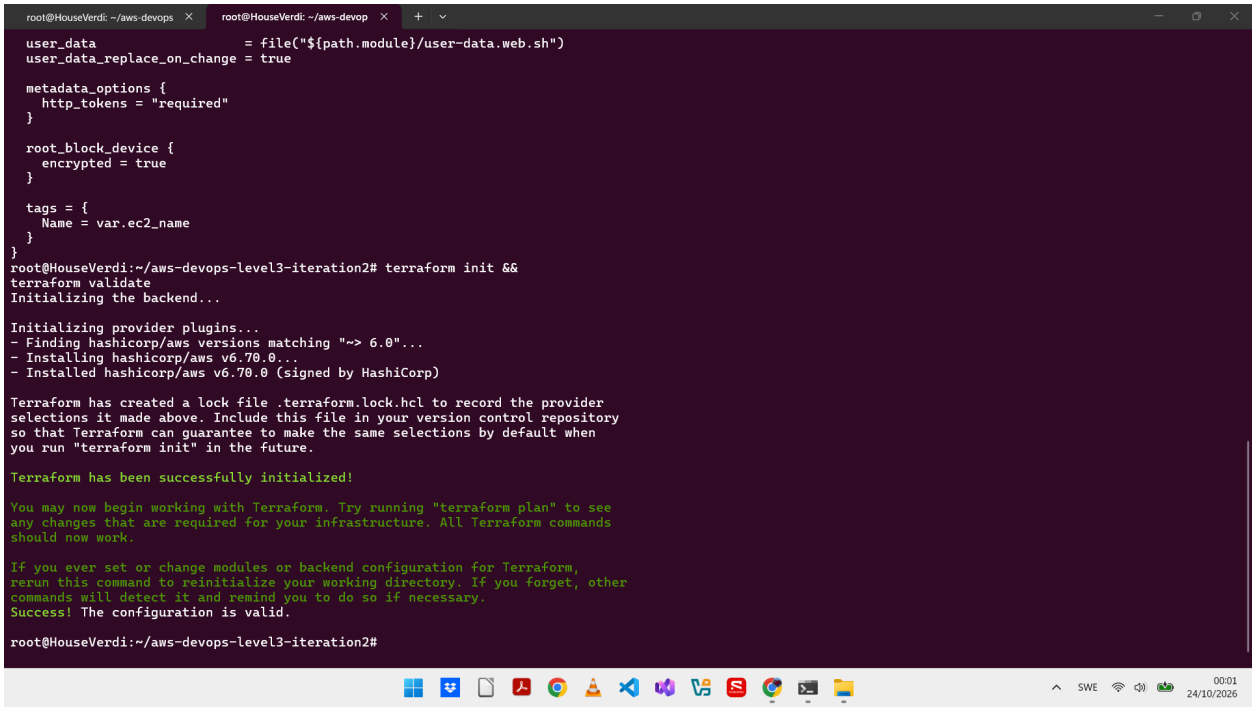


Bild 4. Initiering och lyckad validering med AWS provider 6.70.0

Planering och skapande

terraform plan -out=tfplan visade 1 att skapa, 0 att ändra och 0 att ta bort. terraform apply tfplan skapade instansen på 14 sekunder och gav ett publikt DNS-namn. Det betyder att EC2-resursen skapats; cloud-init kontrollerades separat.

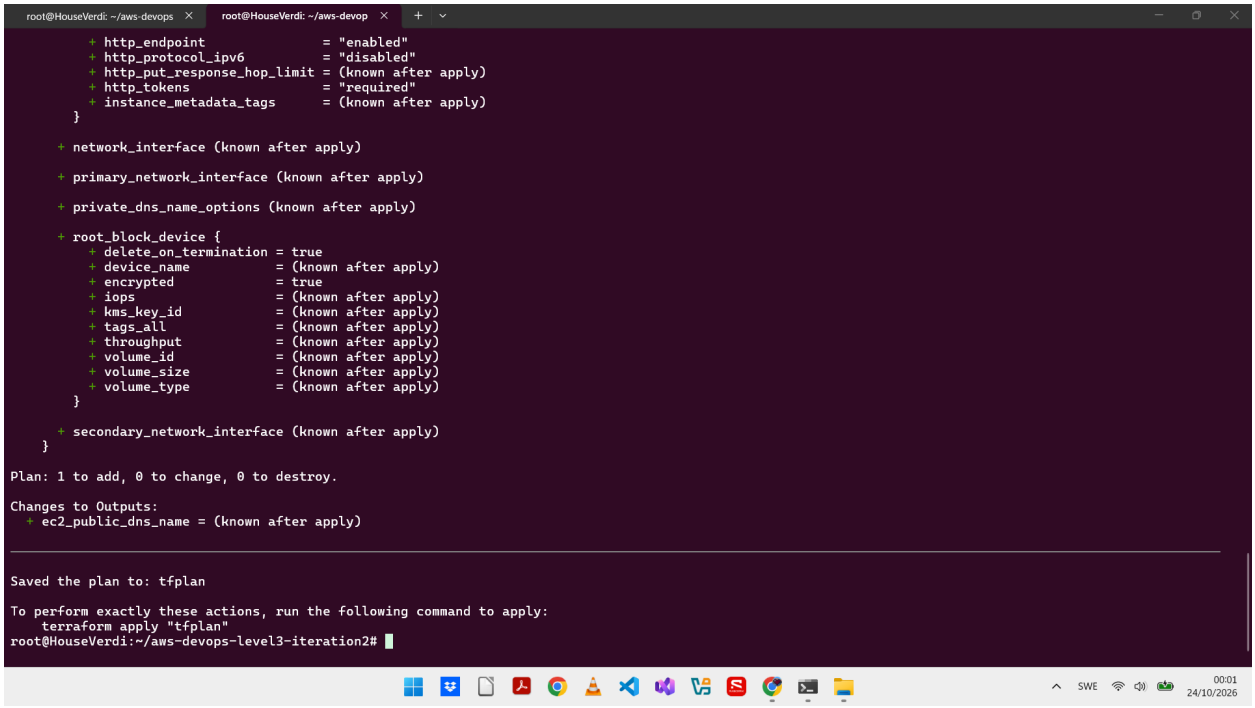


Bild 5. Plan sparad med en ny resurs

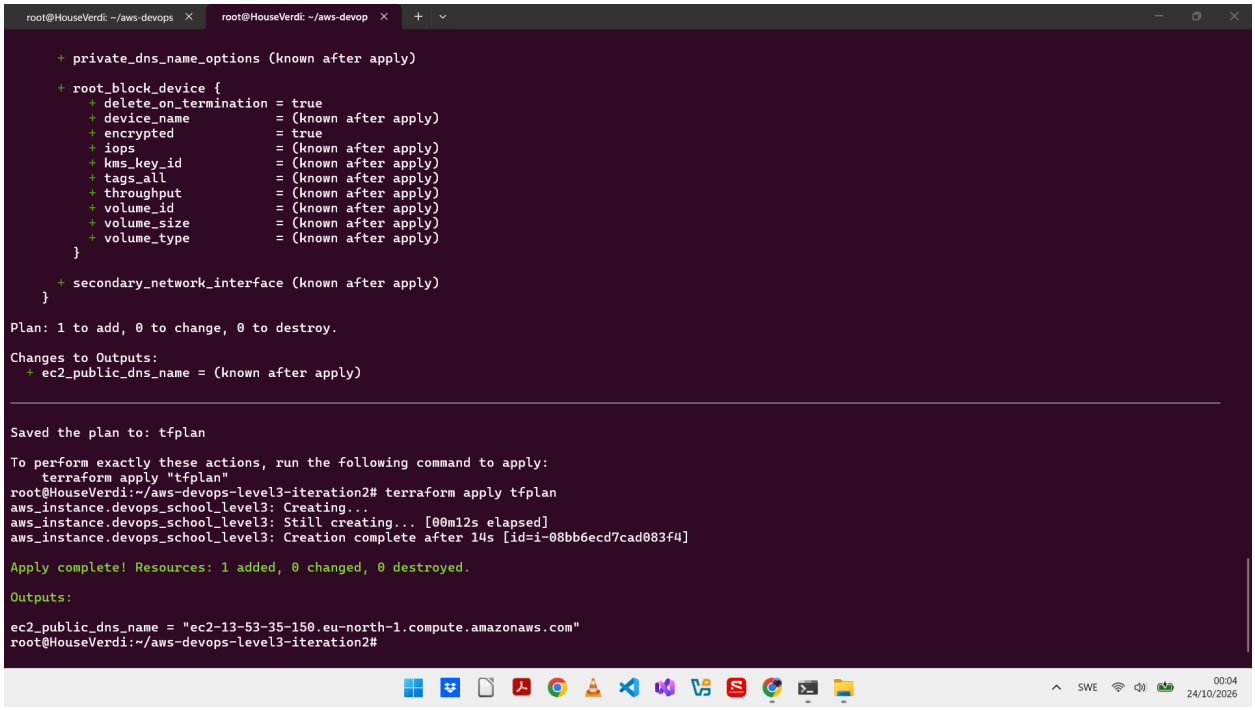


Bild 6. Apply complete med instans-ID och DNS-namn

Verifiering av automatisk installation

Via SSH körde jag sudo cloud-init status --wait, som gav status done. Versions- och tjänstekontrollerna visade Nginx 1.24.0, Git 2.43.0 samt active och enabled. Inga manuella installationskommandon behövdes i SSH-sessionen.

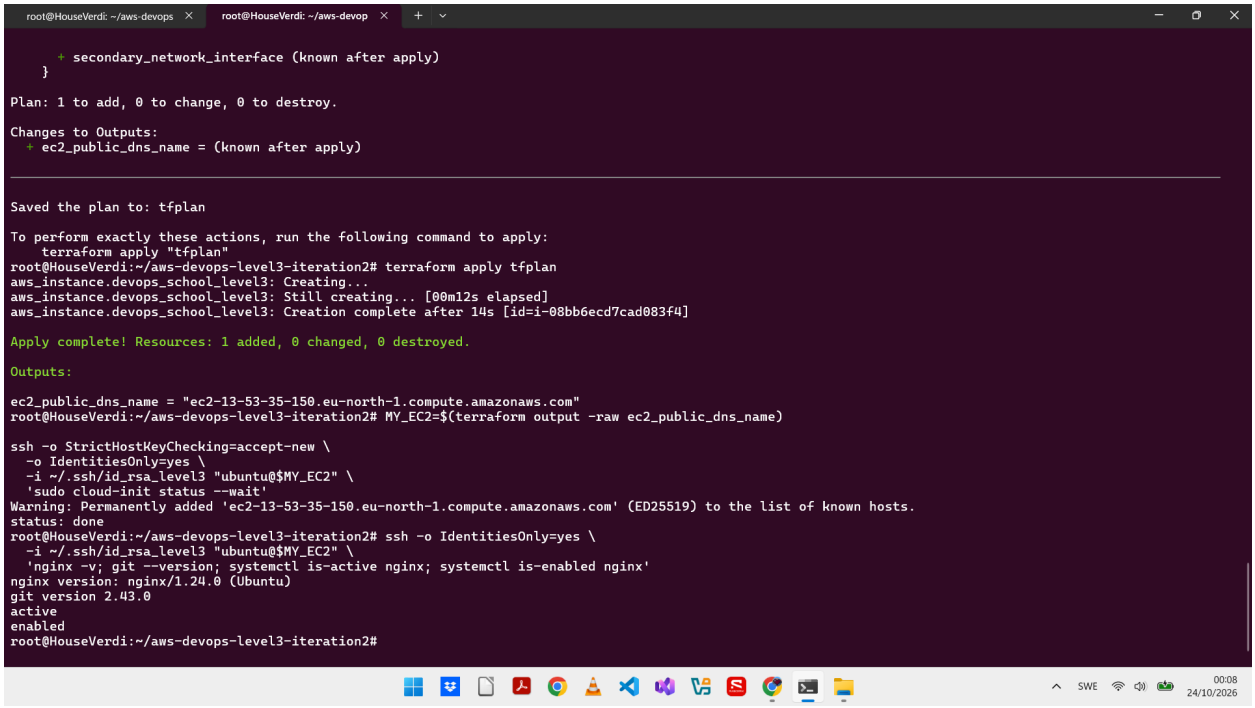


Bild 7. Cloud-init klart samt verifierade versioner och tjänstestatus

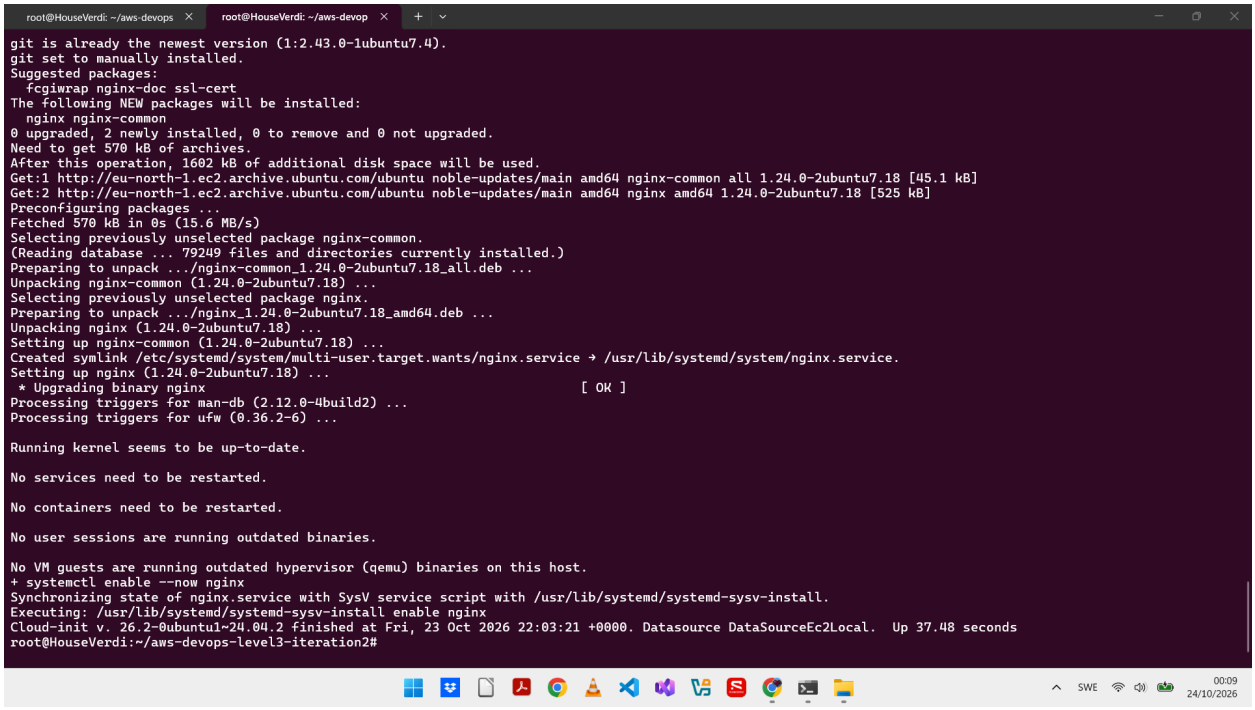


Bild 8. Installationsloggen visar Nginx och avslutad cloud-init

Manuell publicering via SSH

Jag anslöt interaktivt som ubuntu på servern med privat IP 172.31.19.103 och klonade webbplatsen till ~/hello-web. Katalogen innehöll filen heyme på 499 byte, men ingen html-katalog. Jag kopierade därför heyme till /var/www/html/index.html.

sudo nginx -t gav syntax is ok och test is successful. Därefter laddade jag om Nginx.

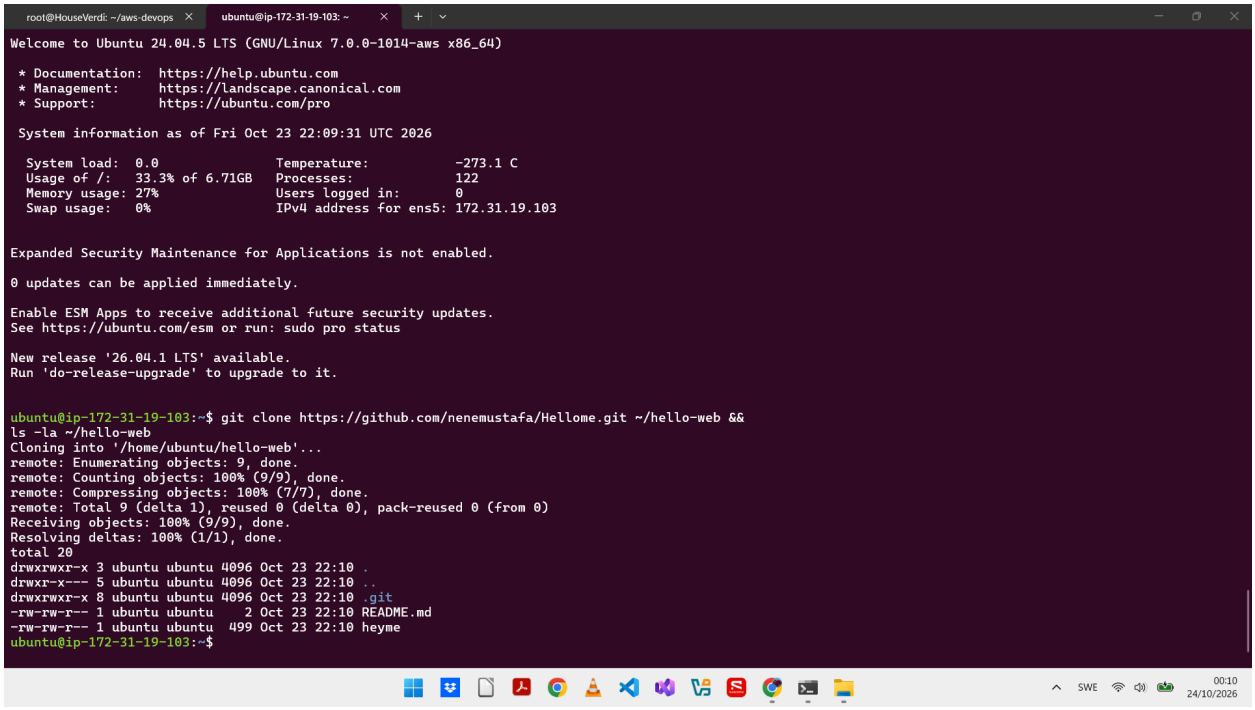


Bild 9. Kloning och kontroll av webbplatsens filer

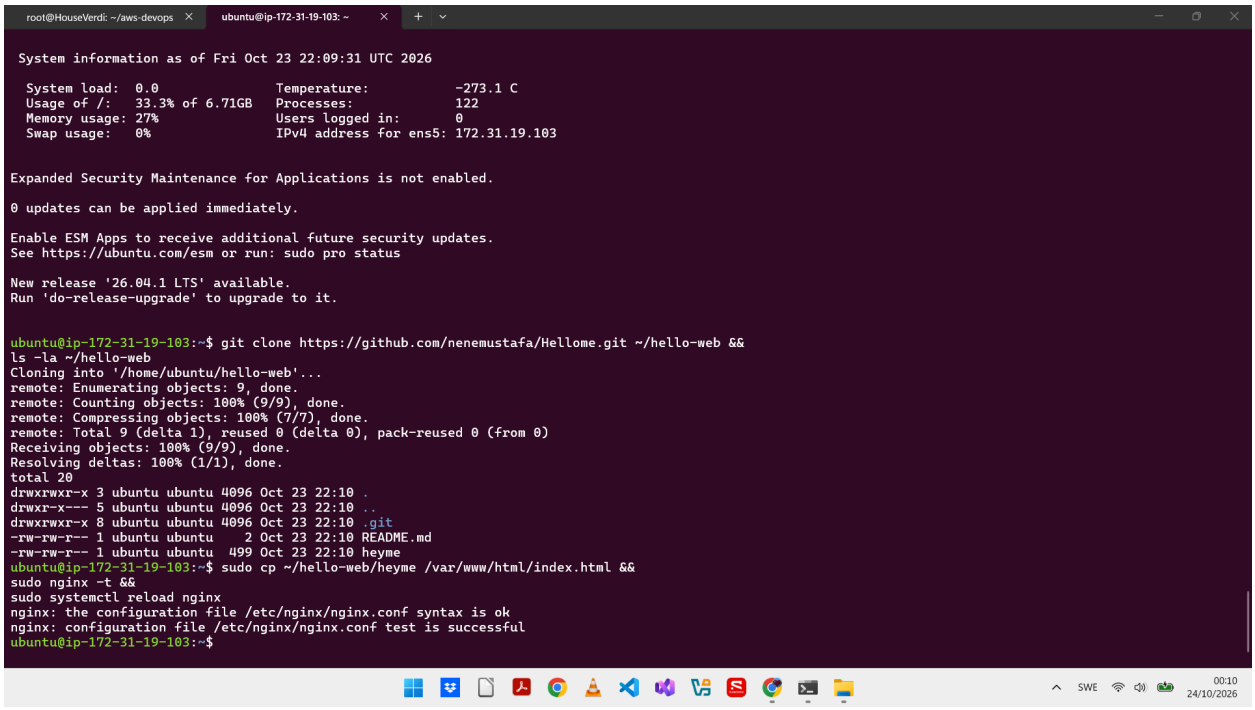


Bild 10. Publicering som index.html och lyckat Nginx-test

Lokalt HTTP test och webbläsare

curl -i http://localhost/ på servern gav HTTP/1.1 200 OK, Content-Type text/html och Content-Length 499. HTML-koden innehöll rubriken Hey me!. Samma sida öppnades från min dator på instansens publika IP.

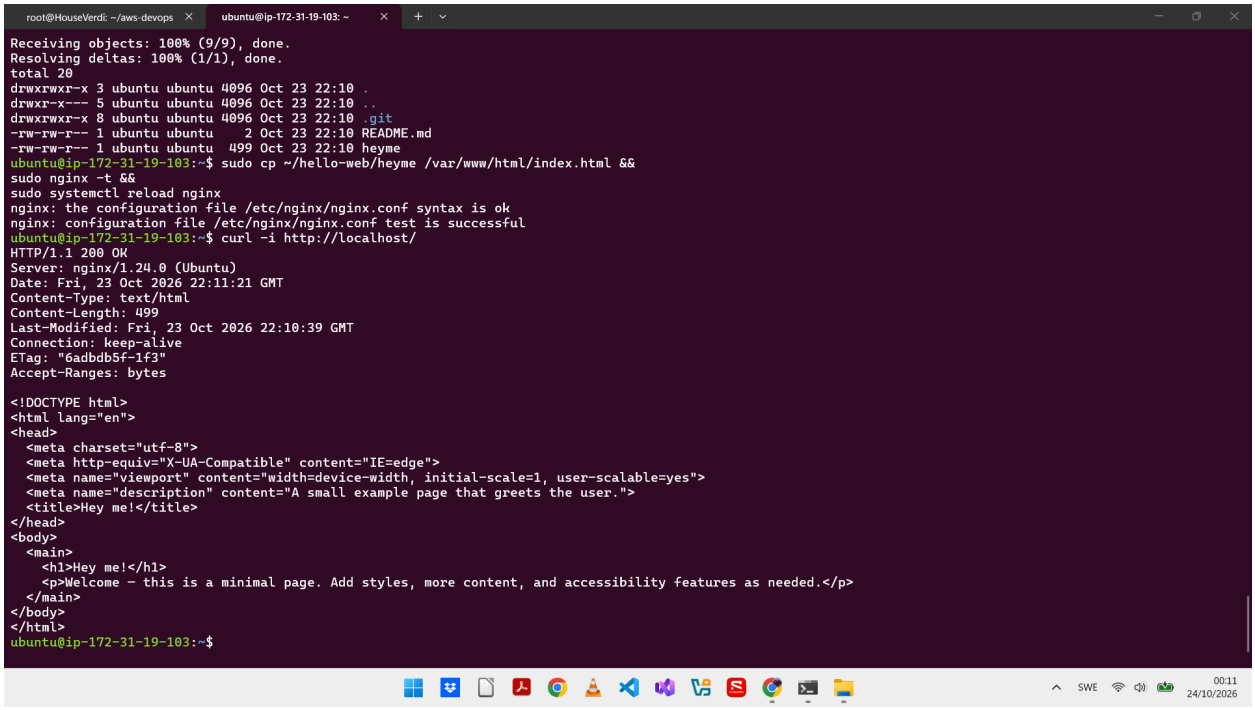


Bild 11. HTTP 200 och sidans HTML på servern

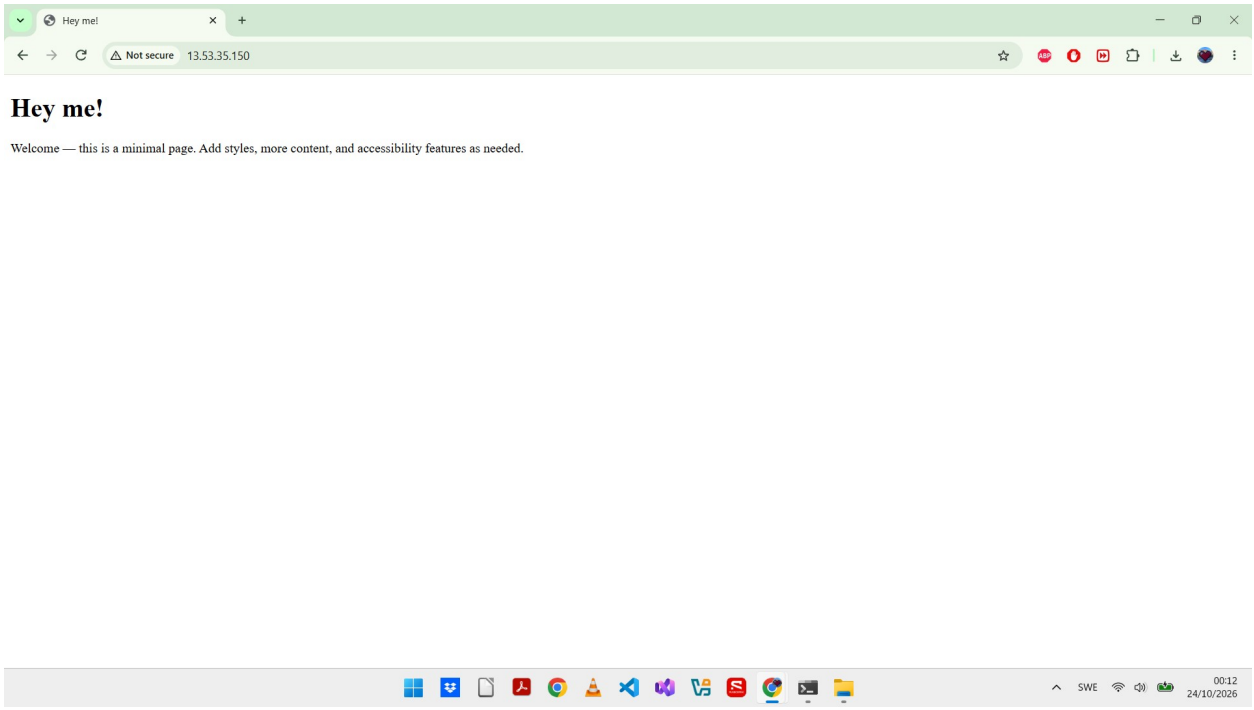


Bild 12. Hey me visas i webbläsaren på 13.53.35.150

Externt test och innehållskontroll

Efter exit från SSH körde jag curl i den lokala WSL-terminalen mot ec2-13-53-35-150.eu-north-1.compute.amazonaws.com. Svaret var HTTP 200 med rätt HTML. Innehållstestet gav YES och slutkod 0.

Jag använde grep -qi eftersom sidans faktiska text Hey me! hade annan skiftlägesform än guidens Hey Me!. pipefail aktiverades för att även fel i curl ska påverka testets slutkod.

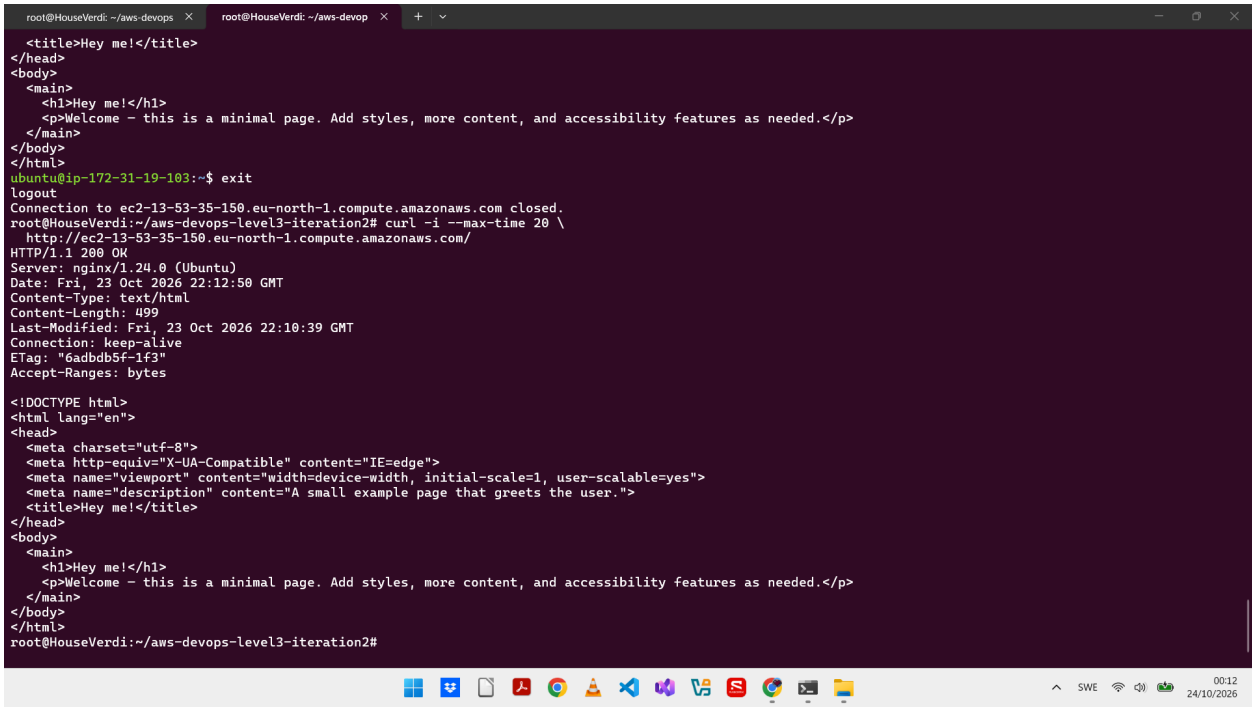


Bild 13. HTTP 200 från den lokala datorn via DNS-namnet

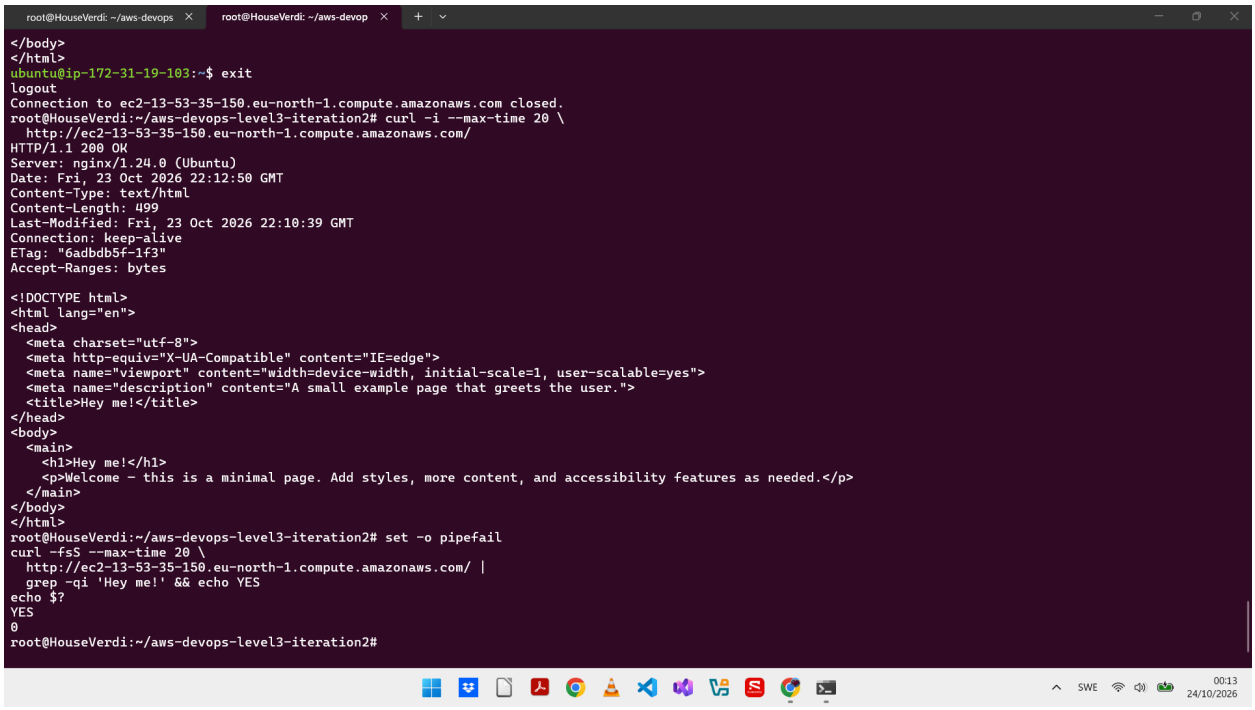


Bild 14. Automatiskt innehållstest med YES och slutkod 0

Terraform kontroll och borttagningsplan

En ny terraform plan gav No changes. Det verifierade att infrastrukturen stämde med konfigurationen efter publiceringen. Denna kontroll omfattar EC2-resursen; webbinnehållet testades separat med HTTP.

Efter att konfiguration, terminalhistorik, serverhistorik och cloud-init-logg sparats skapade jag en borttagningsplan med terraform plan -destroy -out=destroyplan.

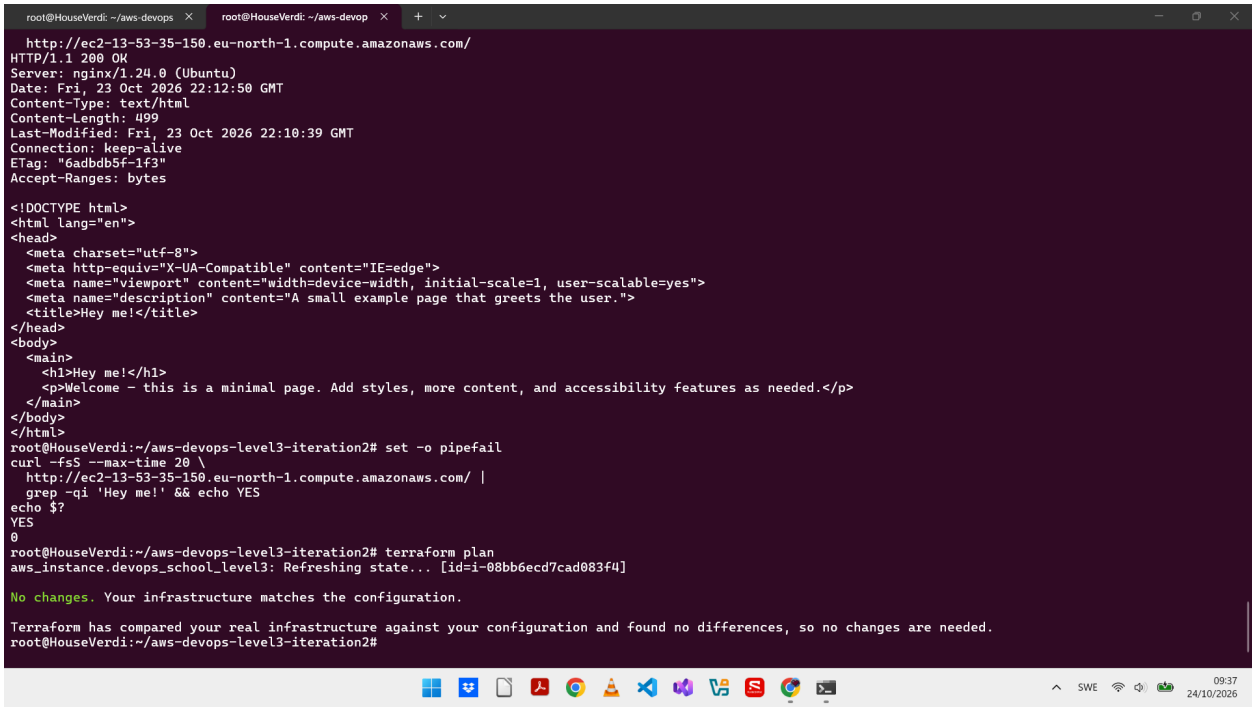


Bild 15. Terraform visar No changes

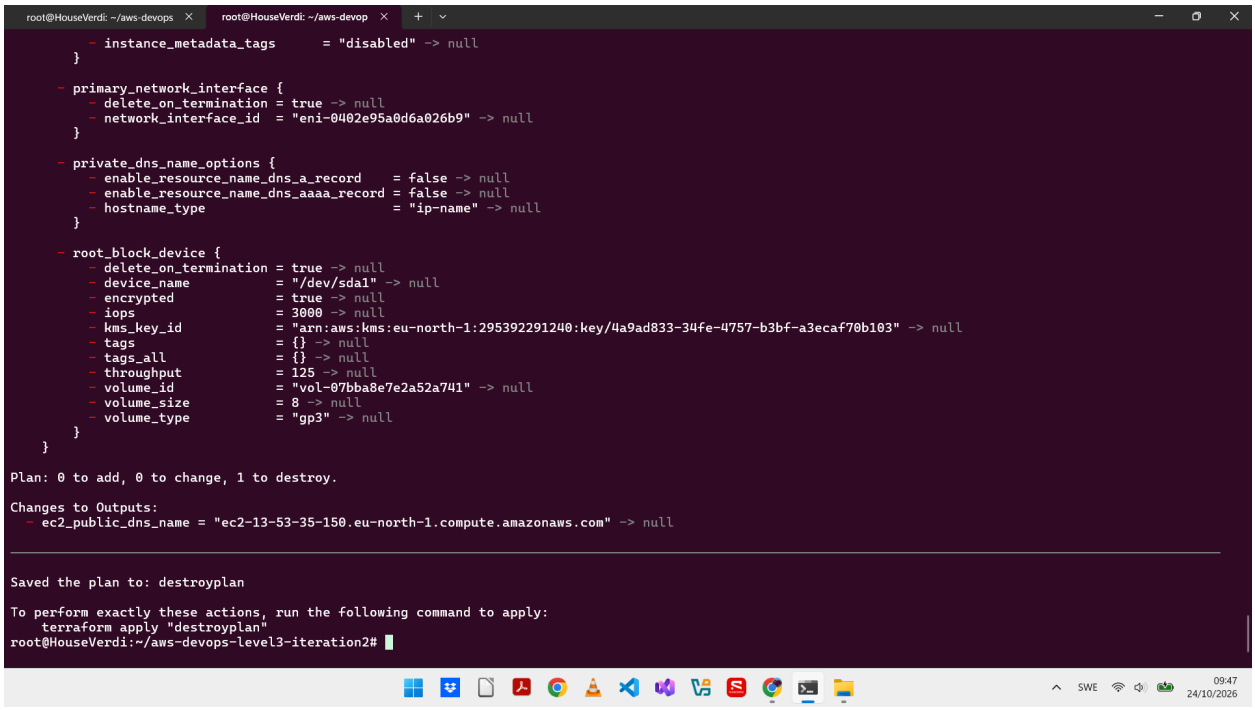


Bild 16. Borttagningsplan med en resurs och krypterad disk

Verifierad borttagning

terraform apply destroyplan tog bort instansen på 20 sekunder och rapporterade 0 added, 0 changed, 1 destroyed. En separat AWS CLI-kontroll gav terminated den 24 oktober 2026 kl. 09.49 svensk tid.

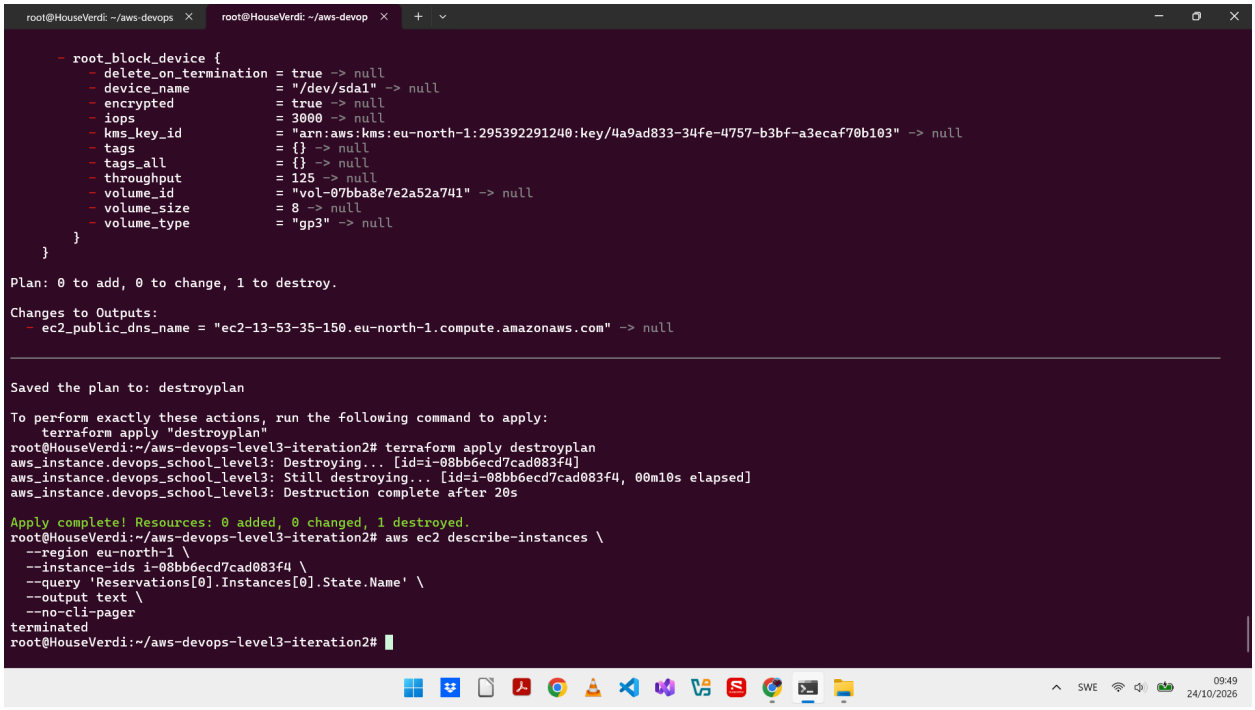


Bild 17. Lyckad borttagning och slutstatus terminated

```
terraform plan -destroy -out=destroyplan
terraform apply destroyplan
aws ec2 describe-instances --region eu-north-1 \
  --instance-ids i-08bb6ecd7cad083f4 \
  --query 'Reservations[0].Instances[0].State.Name' \
  --output text --no-cli-pager
```

Rötdisken hade delete_on_termination=true. Nyckelparet och säkerhetsgruppen låg utanför Terraform och omfattades inte av borttagningen. IP-adressen och DNS-namnet i denna rapport avser webbtester före avslutet.

Anpassningar förbättringar och slutsats

Anpassningar till miljön

Guiden använder yum, service och användaren ec2-user. Eftersom den valda avbildningen var Ubuntu använde jag apt-get, systemctl och användaren ubuntu. Webbrot ändrades till /var/www/html. Filkopieringen anpassades efter att jag granskat det klonade innehållet.

Resultat och förbättringar

Iteration 2 automatiserade paketuppdatering och konfiguration av webbservern med user-data. Publicering av HTML genomfördes manuellt, enligt iterationens upplägg. Git fanns redan i avbildningen; installationsloggen visar att apt kontrollerade paketet och att Nginx installerades som nytt paket.

Nästa förbättring kan vara automatisk publicering av webbplatsen och HTTPS. Nginx var aktiverad för automatisk start, men inget omstartstest utfördes. Låsfilen från terraform init gör det möjligt att återanvända samma provider-version.

Underlag

Referens: Level3 Iteration2 (1)(2)(1).sh. Underlag: level3-2-konfiguration.txt, level3-2-terminal-history.txt, level3-2-server-history.txt, level3-2-cloud-init.log samt infogade skärmbilder. Den lokala historiken innehåller även äldre kommandon; endast sekvensen för iteration 2 används som underlag här.

Förberedelserna började sent den 23 oktober. Skapande och webbtester utfördes efter midnatt den 24 oktober svensk tid. Loggar på servern använder UTC och visar därför fortfarande 23 oktober för dessa moment.

Bilaga Terraform filer

Följande filer återges från den exporterade konfigurationen. Variabelvärdena i terraform.tfvars och skriptet user-data.web.sh hör till separata filer.

providers tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

variables tf

```
variable "aws_region" {
  type = string
}

variable "ami_id" {
  type = string
}

variable "subnet_id" {
  type = string
}

variable "main_sg_id" {
  type = string
}

variable "ec2_type" {
  type = string
}

variable "ec2_name" {
  type = string
}

variable "keypair" {
  type = string
}
```

Bilaga instans och variabelvärden

main tf

```
resource "aws_instance" "devops_school_level3" {
  ami                         = var.ami_id
  instance_type               = var.ec2_type
  subnet_id                   = var.subnet_id
  key_name                    = var.keypair
  vpc_security_group_ids      = [var.main_sg_id]
  associate_public_ip_address = true

  user_data                   = file("${path.module}/user-data.web.sh")
  user_data_replace_on_change = true

  metadata_options {
    http_tokens = "required"
  }

  root_block_device {
    encrypted = true
  }

  tags = {
    Name = var.ec2_name
  }
}
```

outputs tf

```
output "ec2_public_dns_name" {
  description = "Public DNS name of the EC2 instance"
  value       = aws_instance.devops_school_level3.public_dns
}
```

terraform tfvars

```
aws_region = "eu-north-1"
ami_id     = "ami-06fb1802e2b757f09"
subnet_id  = "subnet-05d4dca4820d2eb94"
main_sg_id = "sg-0fb87d0f08a213466"
ec2_type   = "t3.micro"
ec2_name   = "devops-school-level3.tabish-admin"
keypair    = "devops.school.level3.tabish-admin"
```

Bilaga user data och serverkommandon

user data web sh

```
#!/bin/bash
set -euxo pipefail
export DEBIAN_FRONTEND=noninteractive

apt-get update
apt-get upgrade -y
apt-get install -y nginx git

systemctl enable --now nginx
```

Manuell publicering på servern

```
git clone https://github.com/nenemustafa/Hellome.git \
  ~/hello-web && \
  ls -la ~/hello-web
sudo cp ~/hello-web/heyme /var/www/html/index.html && \
  sudo nginx -t && \
  sudo systemctl reload nginx
curl -i http://localhost/
exit
```

Kontroll av automatisk installation

```
sudo cloud-init status --wait
nginx -v
git --version
systemctl is-active nginx
systemctl is-enabled nginx
sudo tail -n 60 /var/log/cloud-init-output.log
```

Kontrollerna ovan kördes på servern genom SSH från den lokala terminalen. Cloud-init-loggen sparades separat i sin helhet.